UADC PoC Test Execution Report

Generated from repository Markdown sources. Paths are repository-relative unless otherwise stated.

Test Execution Report

Test Execution Report

Summary

Date: 2026-07-06

Scope:

- EN 16931 LHM-driven syntax binding conversion.
- Structured CSV generation.
- xBRL-CSV JSON metadata generation linking structured CSV columns to taxonomy concepts.
- Arelle validation of generated xBRL-CSV metadata.
- Structured CSV to UBL Invoice XML reverse conversion.
- UBL 2.1 schema validation of regenerated round-trip XML.
- Round-trip artifact generation under `tests/roundtrip`.
- BIS Billing 3 Invoice example conversion.
- LHM semantic path and class/element checks.
- xBRL taxonomy generation regression check.

Result:

```
PASS
```

Test Environment

Working directory:

```
UADC_PoC
```

Python used by Codex runtime:

```
C:\Users\nobuy\.cache\codex-runtimes\codex-primary-runtime\dependencies\python\python.exe
```

Project Python path shown in user-facing guides:

```
C:\Users\nobuy\AppData\Local\Programs\Python\Python310\python.exe
```

Input Data

PoC minimal invoice:

```
samples/input/openpeppol_ubl_invoice_minimal.xml
```

BIS Billing 3 Invoice examples:

```
samples/input/bis-billing3-examples/
```

Included Invoice files:

- `Allowance-example.xml`
- `base-example.xml`
- `base-example_profile02.xml`
- `base-negative-inv-correction.xml`
- `sales-order-example.xml`

- `vat-category-E.xml`
- `vat-category-O.xml`
- `Vat-category-S.xml`
- `vat-category-Z.xml`

CreditNote examples are excluded from this Invoice test scope.

Generated Round-Trip Artifacts

Artifacts are generated under:

```
tests/roundtrip/
```

Directory layout:

```
tests/roundtrip/<dataset>/
  source_xml/
  structured_csv/
  metadata_json/
  roundtrip_xml/
```

Generated cases:

- `openpeppol-minimal`: 1 case.
- `bis-billing3-examples`: 9 cases.

Total:

```
10 round-trip cases
```

Executed Commands

Taxonomy generation:

```
& $python .\tests\test_xbrlgl_generator_uadc_lhm.py
```

Round-trip artifact and metadata verification:

```
& $python .\tests\test_roundtrip_artifacts.py
```

xBRL-CSV metadata validation with Arelle:

```
& $python .\tests\test_xbrl_csv_metadata_arelle.py
```

Round-trip XML validation with UBL 2.1 schema:

```
& $python .\tests\test_roundtrip_xml_ubl_schema.py
```

Syntax binding reverse conversion:

```
& $python .\tests\test_syntax_binding_reverse.py
```

OpenPeppol structured conversion:

```
& $python .\tests\test_openpeppol_invoice_conversion.py
```

BIS Billing 3 example conversion:

```
& $python .\tests\test_bis_billing3_examples_conversion.py
```

LHM checks:

```
& $python .\tests\test_lhm_semantic_paths.py
& $python .\tools\check_lhm_class_element.py .\specs\lhm\EN16931_CIUS_Invoice_LHM.csv
& $python .\tests\test_lhm_hierarchical_csv_layout.py
```

Execution Results

Taxonomy generation:

```
ok: XBRL-GL generator accepted UADC LHM CSV
```

Round-trip artifacts:

```
openpeppol-minimal: built 1 round-trip case(s)
bis-billing3-examples: built 9 round-trip case(s)
ok: built 10 round-trip artifact(s) under tests/roundtrip
ok: checked 10 round-trip artifact case(s)
```
xBRL-CSV metadata validation:

```
ok: validated 10 xBRL-CSV metadata file(s) with Arelle
```
UBL 2.1 schema validation:

```
ok: validated 10 round-trip XML file(s) against UBL 2.1 Invoice schema
```
BIS Billing 3 examples:

```
ok: converted and checked 9 BIS Billing 3 Invoice example(s)
```
Reverse conversion:

```
ok: reversed out/hierarchical/en16931_lhm_hierarchical.csv to out/reverse/en16931_reverse_invoice.xml
```
OpenPeppol structured conversion:

```
ok: wrote and checked out/structured/openpeppol_ubl_invoice_structured.csv
```
LHM checks:

```
ok: checked 197 semantic path(s)
ok: checked 197 LHM row(s)
ok: wrote and checked out/hierarchical/en16931_lhm_hierarchical.csv
```

Verification Points

Structured CSV:

- Source XML creates hierarchical structured CSV without conversion errors.
- `dInvoice` and repeated `d*` dimension columns are present.
- LHM `lhm_level` is used as the effective hierarchy for Structured CSV and taxonomy modeling.
- Non-repeating BGs such as Seller and Buyer have blank `lhm_level` and are not emitted as dimension columns.
- `InvoiceNumber`, `DocumentCurrencyCode`, invoice line identifiers, and VAT category values are extracted.
- BT-110 and BT-111 are separated by `TaxAmount/@currencyID` predicate.

Metadata JSON:

- One metadata JSON file is generated for each structured CSV.
- The metadata is xBRL-CSV metadata, not a UADC-specific mapping format.
- `documentInfo.taxonomy` references generated `plt-oim` as the xBRL-CSV taxonomy entry point.
- `tables.structured.url` references the generated structured CSV.
- `tableTemplates.structured.dimensions["plt:d_en16931_Invoice"]` maps to `$dInvoice`.
- `tableTemplates.structured.columns.InvoiceNumber.dimensions.concept` maps to `en16931:InvoiceNumber`.
- `tableTemplates.structured.columns.DocumentCurrencyCode.dimensions.concept` maps to `en16931:DocumentCurrencyCode`.
- Arelle `loadFromOIM` validation passed for all 10 generated metadata files.

Round-trip XML:

- Round-trip XML is generated from structured CSV.
- Root-level `cbc:ID` matches CSV `InvoiceNumber`.
- Representative amount elements carry `currencyID` derived from `DocumentCurrencyCode`.
- Allowance and charge branches materialize `cbc:ChargeIndicator=false` or `true`.
- UBL-required non-BT support elements are added where needed, including `cac:TaxScheme/cbc:ID` and default price allowance `cbc:ChargeIndicator=false`.
- UBL 2.1 schema validation passed for all 10 regenerated XML files.

Currency handling:

- BT-5 `DocumentCurrencyCode` is used for ordinary invoice amount `currencyID`.
- BT-6 `TaxAccountingCurrencyCode` is used for tax accounting currency TaxTotal branch.
- `Allowance-example.xml` confirms `TaxAccountingCurrencyCode=SEK`.
- `Allowance-example.xml` confirms `InvoiceTotalVatAmountInAccountingCurrency=9324.00`.

Taxonomy:

- `plt-oim-2026-07-05.xsd` is the xBRL-CSV taxonomy schema.
- `plt-all-2026-07-05.xsd` is not generated.
- `en16931-content-2026-07-05.xsd` is not generated.
- Module schemas reference `../gen/gl-gen-2026-07-05.xsd`.
- Non-repeating BGs such as Seller and Buyer are not emitted as `d_*` dimension concepts.
- Local taxonomy structure validation passed with `tests/validate_taxonomy.py`.
- Arelle validation passed for `out/taxonomy/plt/plt-oim-2026-07-05.xsd`.

Taxonomy validation commands:

```
& $python .\tests\validate_taxonomy.py
& 'C:\Users\nobuy\AppData\Local\Programs\Python\Python310\Scripts\arelleCmdLine.exe' `
  --file .\out\taxonomy\plt\plt-oim-2026-07-05.xsd `
  --validate
```

Notes

The regenerated XML is not expected to be byte-for-byte identical to the source XML. The purpose of the round-trip tests is to confirm that bound semantic values, hierarchy dimensions, taxonomy metadata, and required syntax-derived values are represented consistently.

Known expected differences may include:

- XML declaration formatting.
- Namespace declaration placement.
- Indentation.
- Unbound XML content.
- UBL-required support elements that are not EN 16931 BT values.